

claude-windows / 96963dab-d38f-488c-af44-530ddbefe1

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\96963dab-d38f-488c-af44-530ddbefe1\subagents\workflows\wf_dd531439-5e0\agent-ac794e20b04aff552.jsonl`
- SHA-256: `513294df6a36247bbcdfb036fb5b3f6d91cb489d93dff8bc59a814288bde5fa6`
- Source modified: `2026-07-08T06:38:00+00:00`
- Imported at: `2026-07-08T15:59:27+00:00`
- project: `wf_dd531439-5e0`
- session_id: `96963dab-d38f-488c-af44-530ddbefe1`
```

Transcript

1. user (2026-07-08T06:36:06.954Z)

You are reviewing an UNCOMMITTED diff in an isolated git worktree at C:/Users/avidu/AppData/Local/Temp/claude/wt524-969 (branch feat/524-l4-primary, based on origin/master).

To see the change: cd to the worktree and run `git --no-pager diff` (working tree vs HEAD) and `git --no-pager diff --stat`; read the new files docs/COMPUTE_DECOUPLED_SERVING/L4_PRIMARY_TOPOLOGY.md and tests/test_video_id_trust.py directly. Read surrounding code as needed ? do NOT limit yourself to diff hunks.

CONTEXT ? GitHub issue #524 (epic): collapse the pipeline onto the L4 worker; shrink control-plane to a thin router. This PR delivers (1) a design doc resolving the compute-topology question (recommend co-located scale-to-zero Cloud Run Jobs + bucket upload; REJECT upload-onto-L4 and persistent-GPU with a cost model), and (2) "win #2": the control-plane drain trusts the SERVER-DERIVED submit-time md5 (jobs.video_id ? submit_time_video_id, #484 ? GCS metadata md5 converted to hex by backend/storage/gcs.py::md5_hex) instead of re-hashing the fetched clip, gated by deployment.trust_submit_video_id (default True) and guarded by orchestration._looks_like_video_id (strict 32-char lowercase hex). Win #1 (validation-on-L4) is deliberately NOT in this PR ? it is delivered by the separate open branch feat/521-compile-on-l4 (phase_placement.validation); duplicating it is forbidden.

Report ONLY concrete, actionable findings with file:line. No praise, no restating the design. For each: what breaks, the exact failing input/state, and the fix. If a dimension is clean, say so explicitly.

A prior reviewer raised this finding on the REPO CONVENTIONS / DISCIPLINE dimension:

SEVERITY: low

FILE: docs/COMPUTE_DECOUPLED_SERVING/L4_PRIMARY_TOPOLOGY.md:23

SUMMARY: Doc's own [verified]-tagged code anchors point at the wrong lines because THIS PR shifts them ? a same-commit honesty defect that violates the doc's stated tagging convention.

CLAIMED FAILURE: §0 (line 23) cites backend/orchestration.py:748 as acquire_local_input and :751 as the compute_video_id re-hash; §10 (line 218) cites :741-763 as the re-hash. After this PR's insertion, orchestration.py:748 is the compat-shim `if config is not None and not isinstance(config, AppConfig):`, :751 is `req = db`, and 741-763 is the docstring/import/shim region. The real acquire_local_input is now :771 and the real compute_video_id re-hash is :798. Anyone following these [verified] anchors lands on unrelated code.

PROPOSED FIX: Update §0 to acquire_local_input at :771 and the re-hash at :798, and §10 to :798 for the re-hash (or drop brittle absolute line numbers in favor of symbol names, which is what the doc already does for most anchors).

Adversarially VERIFY it against the real code/doc in

C:/Users/avidu/AppData/Local/Temp/claude/wt524-969. Default to REFUTED unless you can construct the concrete failing input/state and confirm it in the actual code. A finding is CONFIRMED only if it is a real defect a maintainer must fix (not a style preference, not already-handled, not a misread). State the verdict and the evidence (file:line you checked).

2. assistant (2026-07-08T06:36:10.996Z)

I'll verify this finding against the actual code and doc in the worktree.

3. user (2026-07-08T06:36:21.300Z)

warning: in the working copy of 'docs/COMPUTE_DECOUPLED_SERVING/L4_PRIMARY_TOPOLOGY.md', LF will be replaced by CRLF the next time Git touches it

warning: in the working copy of 'tests/test_video_id_trust.py', LF will be replaced by CRLF the next time Git touches it

backend/api/job_worker.py	5 +
backend/config/models.py	10 +
backend/orchestration.py	51 +---
.../L4_PRIMARY_TOPOLOGY.md	225 +-----
docs/COMPUTE_DECOUPLED_SERVING/README.md	2 +
tests/test_config_deployment.py	2 +
tests/test_video_id_trust.py	173 +-----
7 files changed, 466 insertions(+), 2 deletions(-)	

4. user (2026-07-08T06:36:22.521Z)

```
1      # L4-primary pipeline topology ? collapse the pipeline onto the worker, shrink the
control-plane to a thin router
2
3      > **Status:** `DRAFT` ? **owner-review** (recommends *against* the epic's proposed
direction #1; see §0/§8) . **Owner:** Avi . **Created:** 2026-07-08 . **Last updated:**
2026-07-08
4      > **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when
DONE)
5      > **Issue:** [#524](https://github.com/Khelsutra/badminton-highlight-indexer/issues/524)
(epic) . **Builds on:** [#521](https://github.com/Khelsutra/badminton-highlight-
indexer/issues/521) (`phase_placement`) . **Motivated by:**
[#520](https://github.com/Khelsutra/badminton-highlight-indexer/issues/520) (the tracing that
surfaced the data-movement finding)
6      > **Relation to existing docs:** extends
[COMPUTE_DECOUPLED_SERVING/README.md](./README.md) (ADR-014) and [`PHASE_PLACEMENT.md`
(#521)](https://github.com/Khelsutra/badminton-highlight-indexer/blob/feat/521-compile-
on-l4/docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md); it does **not** supersede them ? it
answers the *topology* question they left open and sequences the migration on top of
`phase_placement`.
7      > **Honesty note:** each claim is tagged `[verified]` (checked against code/runlog),
`[researched]` (needs sign-off / external pricing I could not fetch live), or `[design]`
(proposed). Not legal/financial advice.
8
9      ---
10
11     ## 0. TL;DR
12
13     The 2026-07-08 live CUJ moved one 1.83 GiB clip **3x** (upload?bucket, bucket?control-
plane to validate, bucket?worker to detect) and pinned the CPU-only 2-vCPU control-plane for
**~16 min** of heavy work (validation 183 s + stitch ~12m55s). The epic proposes fixing this by
**uploading directly onto the L4** and async-copying to the bucket. **This design recommends
*against* that direction** and quantifies why: a same-region bucket?compute transfer inside GCP
is **LAN-fast and not billed as internet egress** `[researched]`, so the round-trip that upload-
onto-L4 saves is worth **~seconds and ~$0**, while keeping an L4 up to *receive* a *minutes-
long** upload burns **~175 s of idle GPU per run (~$0.08?0.16)** `[design]` and *opens* a
durability exposure window instead of closing one.
14
15     The cheaper, more durable, already-ratified path is the opposite: **keep the upload
streaming to the cheap bucket** (the #480 design ? the durable copy exists *before* the L4 ever
starts), and **collapse validate + detect + stitch onto the scale-to-zero Cloud Run L4 Job** ?
which is exactly what **#521's `phase_placement` already wires as a config flip**. The residual
"~30 s pre-validation gap" is then killed not by moving the *upload* but by the control-plane
**never downloading the file at all** (it delegates validation to the L4 per #521, and trusts
the already-known submit-time md5 per this epic's win #2 ? no re-hash). Net data movement drops
```

from 3x to the theoretical floor: ****1 client upload + 1 LAN-fast intra-region pull****, with ****zero idle-GPU cost**** and ****zero exposure window****.

16

17 ****Actionable now (this PR):**** the design doc + ****win #2**** (drop the redundant control-plane re-hash, config-guarded, reversible). ****Win #1**** (validation-on-L4) is ****delivered by #521**** ? this doc does not duplicate its knob. A persistent GPU (GCE VM / Cloud Run Service-with-GPU) is ****rejected at current volume**** on a quantified cost basis and left as an owner-gated future ADR.

18

19 **## 1. Problem & goal**

20

21 ****Why now.**** Tracing the live CUJ (``runlogs/DEPLOY_REPORT_cloud-serving_validation-latency_2026-07-07.md`` §9 ``[verified]``) produced two findings:

22

23 1. ****Data is moved 3x per run**** ``[verified]``: (a) upload streams straight to ``gs://?/uploads/?`` (never touches control-plane disk ? the #480/#471 OOM fix, ``backend/api/uploads.py``), computing the md5 incrementally as it streams; (b) the control-plane re-downloads the whole clip to scratch to validate ? ``_execute_processing`` ? ``storage.acquire_local_input(gs://?)`` at ``backend/orchestration.py:748``, then a ****redundant re-hash**** ``compute_video_id`` at ``:751``; (c) the worker downloads it a third time to detect (``[worker]`` pulled ?). The unlogged ****~30 s pre-validation gap**** is finding (b).

24 2. ****The control-plane is the scaling wall**** ``[verified]``: validation (183 s) ****and**** stitch (~12m55s, ~67 s/clip 4K re-encode, no NVENC) both run in-process on the 2-vCPU control-plane, serialized by ``_LOCAL_COMPUTE_LANE`` ? ~16 min of heavy CPU per run ? the API saturates past ~5 concurrent users.

25

26 ****The concrete outcome we want.**** The L4 worker becomes the primary compute for the whole pipeline; the control-plane shrinks to a ****thin router**** (auth · dispatch · DB · SPA + job status · signed-URL download) that does ****no heavy decode/encode**** and, ideally, ****never pulls the full file****. Every move must be ****reversible by config**** with a ****bucket-mediated fallback****, and land as small PRs on top of ``origin/master``.

27

28 ****The hard question this doc must resolve (compute topology).**** "Upload directly to the L4" needs an ****inbound endpoint on the GPU box****, but the current L4 is a Cloud Run ****Job**** (batch, no inbound HTTP ? ``backend/compute/cloud_run.py``, dispatched per-request, ``[verified]``). So the epic forces a choice between ****A. Cloud Run Service-with-GPU****, ****B. GCE L4 VM****, and ****C. a hybrid**** presign/redirect. §4 answers it with numbers.

29

30 ****Read to get current:**** ``[PHASE_PLACEMENT.md`` (#521)](https://github.com/Khelsutra/badminton-highlight-indexer/blob/feat/521-compile-on-l4/docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md) ? the ``phase_placement`` knob this sequences on, ``[README.md]``(``./README.md``) §2 decisions ****D4/D7/D16**** (the ratified constraints that bound the options), and the runlog §9 (the evidence).

31

32 **## 2. Decisions locked**

33

#	Decision	Source / date	Implication
L1	**Keep upload ? bucket** (streaming, #480). The durable copy exists **before** any compute runs; the bucket **is** the "reliable co-located disk" the epic wants ? no VM needed for durability. this doc <code>`[design]`</code> + #480 <code>`[verified]`</code> No exposure window; upload cost stays on the cheap I/O path.		
L2	**Collapse validate + detect + stitch onto the scale-to-zero Cloud Run L4 Job** via <code>`phase_placement`</code> (#521), not onto a persistent GPU. Honors **D7** (scale-to-zero) + **D16(a)** (Jobs, not Services). #521 <code>`[verified]`</code> + D7/D16 The "L4-primary" vision is realized by **config flips** , not a rearchitecture.		
L3	**Kill the ~30 s gap by *not downloading to the control-plane*, not by uploading onto the L4.** Compose #521's validation-delegation (CP skips the validator suite) with win #2 (CP trusts the submit-time md5 ? no re-hash). this doc <code>`[design]`</code> The CP stops pulling the full file when validation is delegated; movement ? 3x?2x.		
L4	**Reject persistent-GPU (GCE VM / Service-with-GPU) at current volume.** Breakeven vs scale-to-zero is a **~51 % on-demand / ~31 % spot duty cycle** (? 2,600?4,300 runs/mo sustained) <code>`[design]`</code> ; we are at pilot volume. Revisit only behind a **new ADR** at sustained load. §4.2 cost model <code>`[design]`</code> Preserves per-run billing (D7/D8); avoids ~\$382?637/mo idle.		

```

40 | L5 | **Reject "upload onto the L4."** It saves a ~free, seconds-fast intra-region LAN
pull at the cost of ~175 s idle GPU/run **and** opens a durability window. | §4.3 `[design]` |
The epic's proposed direction #1 is not adopted; owner sign-off requested (§8). |
41 | L6 | **Win #2 (drop the re-hash) is config-guarded + reversible**
(`deployment.trust_submit_video_id`, default on) and produces a **byte-identical `video_id`**
(the GCS `md5Hash` over the same bytes = `compute_video_id`). | win #2 `[verified]` | Zero-
regression; one-flag rollback. |
42
43 *(IDs are `L#` to avoid colliding with README.md's `D#`. These refine ? never override ?
D4/D7/D16.)*
44
45 ## 3. Foundation ? the ratified constraints this must live inside
46
47 The topology is **not a blank slate** ? ADR-014 (README.md §2) already ratified the
serving substrate. The options in §4 are scored **against** these, and any option that breaks
one needs a new ADR, not just this doc:
48
49 | Ratified decision | What it pins | Consequence for #524 |
50 |---|---|---|
51 | **D4** | In cloud-serving, **stitch runs on Cloud Run** (co-located, metered). | The
live system's CP-stitch is a *deviation* from D4; #521 *realizes* D4. #524 is mostly
**completing an already-ratified decision**, not inventing one. |
52 | **D7** | **Scale-to-zero, no warm pool** (a ~$500/mo idle GPU "would break per-video
billing"). | Any persistent-GPU option (B, Service-with-min-instances) **contradicts D7** ? new-
ADR territory (§4.2). |
53 | **D16(a)** | **Cloud Run Jobs** (not Services-with-GPU), **bucket-poll** for
completion. | Option A (Service-with-GPU inbound upload) **contradicts D16(a)**. The Job model
is load-bearing (reuses `JobWaiting`/`claim_due_job`). |
54 | **D16(b)** | Stitch encode = libx264 for parity; **NVENC deferred behind a knob**. |
#521 lands exactly that knob (`cloud_stitch_encoder`), using this CUJ as the cost data D16(b)
said to revisit on. |
55 | **D16(c)** | Input cap hard-reject `< 2 GB`. | The 1.83 GiB CUJ clip is *at* the cap;
the topology must stay sane at 2 GB (upload minutes, pull seconds). |
56
57 **Key reframing this foundation gives us:** the owner's "reliable persistent disk"
requirement is *already met* by the **regional GCS bucket** ? a durable, co-located, cheap
"disk" the upload already writes to directly. The persistent-VM instinct solves a problem
(durable local disk) that the bucket-first design doesn't have.
58
59 ## 4. Design / architecture ? resolving the topology question
60
61 ### 4.1 The measured pipeline (evidence base) `[verified]`
62
63 From runlog §9 (rev `rally-control-plane-00019`, "First CUJ.MP4", 1.83 GiB / 174.17 s /
4K-class):
64
65 | Phase | Wall | Machine | GPU needed? | Notes |
66 |---|---|---|---|---|
67 | Upload (client ? bucket) | **~2m55s** | client uplink | no | 1.83 GiB @ ~84 Mbps ?
**owner bandwidth**, not infra |
68 | Accept ? validate start | ~30 s | control-plane | no | **the gap** = CP re-download to
scratch (`acquire_local_input` + re-hash) |
69 | Validation (#512 one-pass) | **183 s** | control-plane 2 vCPU | no (CPU-bound) |
scene_cut/blur/relevance shared decode |
70 | Dispatch ? worker ready | ~55 s | Cloud Run Job | ? | image pull + GCSFuse +
torch/cuda init (cold start) |
71 | Detection (WASB) | **161.6 s** | L4 (cuda) | **yes** | the only genuinely GPU-bound
phase |
72 | Ingest + finalize | ~4 s | control-plane | no | intervals ? DB |
73 | Compile / stitch | **~12m55s** | control-plane 2 vCPU | no (NVENC would need GPU) |
11× 4K re-encode+watermark |
74
75 **Two facts that drive the whole design:**
76 - **Only *detection* is GPU-bound.** Validation is CPU/decode-bound; stitch is encode-
bound (NVENC *wants* a GPU but libx264 doesn't *need* one). So "collapse onto the L4" is about

```

```

**co-locating CPU work next to the GPU on a box that's already up and already holding the
bytes**, not about the GPU per se.
77     - **Upload dwarfs compute** (175 s upload vs 161.6 s GPU-detect). Anything that keeps a
GPU allocated *during upload* roughly **doubles** the GPU-allocated time for zero compute.
78
79     ### 4.2 Cost model (asia-southeast1, USD, Cloud Billing Catalog API, 2026-07-08)
`[verified]` rates / `[design]` per-run
80
81     Rates pulled live from the Catalog API (`cloudbilling.googleapis.com`, project `gen-
lang-client-0306026826`):
82
83     | Resource | Rate | Source |
84     |---|---|---|
85     | Cloud Run **L4 GPU** (no zonal redundancy) | **$0.00022404 / s** = $0.806/hr | Catalog
`[verified]` |
86     | Cloud Run vCPU (instance/jobs) | $0.0000216 / s | Catalog `[verified]` |
87     | Cloud Run memory | $0.0000024 / GiB.s | Catalog `[verified]` |
88     | ? **L4 Job @ 8 vCPU / 32 GiB (running)** | **$1.705 / hr** ($0.000474/s) | derived
`[design]` |
89     | GCE **g2-standard-4** (4 vCPU/16 GiB + L4), on-demand | **$0.872 / hr** = **-$637 /
mo** | Catalog `[verified]` |
90     | GCE g2-standard-4, **spot/preemptible** | $0.523 / hr = ~$382 / mo | Catalog
`[verified]` |
91     | Cloud Run **GPU Service, warm** (min-instances=1, 4 vCPU/16 GiB) | ? $1.256 / hr =
**-$917 / mo** idle | derived `[design]` |
92     | Balanced PD | $0.11 / GiB.mo | Catalog `[verified]` |
93
94     **Per-run GPU-allocated cost (the term that varies by topology):**
95
96     | Topology | GPU-allocated time / run | GPU cost / run | ? vs today | Concurrency wall?
|
97     |---|---|---|---|
98     | **Today** (detect-only on L4; validate+stitch on CP) | ~222 s (cold+detect+io) |
**~$0.105** | ? | **Yes** ? ~16 min CP CPU/run |
99     | **#521 + win #2** (validate+detect+stitch on L4; upload?bucket) | ~312 s (validate 60
+ detect 162 + stitch 60 + io 30) | **~$0.148** | **+$0.04** | **No** ? CP is thin |
100    | **Upload-onto-L4** (GPU up during upload too) | ~487 s (+175 s idle upload) |
**~$0.231** | **+$0.13** | No, but idle-GPU tax |
101    | **Persistent GCE VM** (on-demand) | billed 24/7 regardless | **$637/mo flat** | breaks
per-run billing | No |
102
103    **Breakeven (persistent vs scale-to-zero):** a persistent on-demand g2 ($0.872/hr) beats
the scale-to-zero Job ($1.705/hr *while running*) only at a **duty cycle ? $0.872/$1.705 ? 51
%** (spot: ? 31 %). At ~312 s/run that is **~2,600?4,300 runs/month sustained, 24/7**
`[design]`. Pilot volume is *handfuls/day*. **Scale-to-zero wins by ~2 orders of magnitude** and
preserves per-run billing (D7/D8). A persistent GPU is also **idle >95 % of the time** at our
volume ? the ~$382?637/mo would be almost pure waste.
104
105    ### 4.3 The three topology options, scored
106
107    **Option A ? Cloud Run Service-with-GPU hosting upload + pipeline.**
108    - *Latency:* scale-to-zero ? same ~55 s cold start as the Job; **min-instances=1**
removes cold start but pins a **-$917/mo warm idle GPU**.
109    - *Cost:* to accept the upload, the GPU Service instance must be **up for the whole 175
s upload** ? the idle-GPU tax of $4.2 on *every* request, plus the warm-idle bill if min-
instances>0.
110    - *Ops/parity:* **contradicts D16(a)** (Jobs, not Services) and **D7** (scale-to-zero) ?
abandons the `JobWaiting`/`claim_due_job`/bucket-poll machinery that's shipped and tested. A
Service also needs request-timeout/concurrency tuning the batch Job sidesteps.
111    - **Verdict: reject.** Buys an inbound endpoint we don't need (the bucket is the inbound
endpoint) at the cost of the idle-GPU tax + two ratified-decision reversals.
112
113    **Option B ? GCE L4 VM (persistent, reliable PD).**
114    - *Reliability:* real, attached, reliable PD ? the owner's stated appeal. But **the
bucket already gives us durable co-located storage** (L1), so the PD solves a non-problem.

```

115 - *Cost:* **\$382?637/mo flat**, >95 % idle at pilot volume; breaks per-run billing (D7/D8). Spot is cheaper but **preemptible** ? directly against the "reliable" motivation.

116 - *Ops:* we now **own a VM** ? patching, disk-fill, crash-recovery, autoscaling-group if we ever need >1. The Job model has none of this.

117 - **Verdict: reject at current volume; owner-gated future ADR** if sustained load crosses the \$4.2 breakeven.

118

119 **Option C ? Hybrid: control-plane presigns/redirects the upload to a just-spun-up L4, pipeline runs there, async copy to bucket.**

120 - *The fatal timing problem:* compute needs the **whole** file, so the L4 cannot start useful work until the upload finishes. Spinning the L4 up *to receive* the upload means the GPU is **allocated and idle for the entire 175 s upload** ? the \$4.2 idle tax, same as A. Upload and compute **cannot overlap**.

121 - *Durability:* introduces an **exposure window** (upload-complete ? async-mirror-confirmed) where the only copy is on L4 local disk; a preemption/crash in that window **loses the source** unless the client re-uploads (see \$4.5).

122 - **Verdict: reject.** It pays the idle-GPU tax *and* takes on a durability window, to save a transfer that is ~free and seconds-fast.

123

124 **Option A? (recommended) ? co-located Jobs, bucket-mediated upload (the L1?L3 design).**

125 - Keep **upload ? bucket** (cheap, bounded-memory, durable-immediately). Keep the **scale-to-zero L4 Job**. Move **validate + stitch** onto the Job via `phase\_placement` (#521). Make the CP **stop downloading** (delegate validation + trust the md5, win #2). The Job pulls the file **once** from the same-region bucket at LAN speed.

126 - *Data movement:* **1 client upload + 1 intra-region LAN pull** ? the floor. (Detection and validation/stitch all run in **one** Job execution, so it's one pull, not two.)

127 - *Cost:* +\$0.04/run GPU vs today, **~16 min CP CPU/run** (the actual scaling win). No idle tax, no warm bill, no VM.

128 - *Durability:* **no exposure window** ? the bucket copy exists before the Job starts.

129 - *Ops/parity:* **honors D4/D7/D16** ; reuses the shipped dispatch/poll path; every phase is a **config flip with an in-process fallback** (#521's `\_dispatch\*\_remote` already logs-and-falls-back to in-process when the box isn't cloud-wired).

130

131 > **The one-line intuition:** the bucket *is* the co-located reliable disk; the GPU should attach only for the ~5 min of compute, pulling from it at LAN speed ? never sit idle for the ~3 min upload.

132

133 ### 4.4 The minimal control-plane surface (the "thin router")

134

135 After A?, the control-plane does **only** I/O-light, decode-free work ? trivially horizontally scalable (the one caveat is SQLite; a Postgres move is out of scope, noted in \$6):

136

137 | Keep on the control-plane | Why it must stay | Code anchor `[verified]` |

138 |---|---|---|

139 | **Cloudflare-Access edge auth** (verify `Cf-Access` JWT) | D9 ? one identity system; direct-hit spoofing guard | `backend/api/auth.py`, gated at `/api/process` |

140 | **Upload endpoint** (streams parts ? bucket) | Bounded-memory, no decode (#480); the durable-copy producer | `backend/api/uploads.py` |

141 | **Dispatch / routing** (submit job, dispatch Cloud Run Job, bucket-poll) | Owns the async job lifecycle (`JobWaiting`/`claim\_due\_job`) | `backend/orchestration.py`, `backend/compute/cloud\_run.py` |

142 | **DB writes** (jobs, videos, `play\_segments`, validation verdict) | The control-plane owns the SQLite DB (worker uses a throwaway scratch DB) | `backend/infrastructure/database\*.py` |

143 | **SPA serving + job status** (`/api/jobs/{id}` polling) | The UX surface | `backend/api/jobs.py`, `backend/ui/` |

144 | **Signed-URL download** (`/api/highlights` 307) | Reel is already in the bucket; CP just signs | `backend/main.py::get\_highlights` |

145 | **~~Validation~~** | ? **L4** via `phase\_placement.validation` (#521) | delegated |

146 | **~~Stitch/compile~~** | ? **L4** via `phase\_placement.compile` (#521) | delegated |

147 | **~~Full-file download + re-hash~~** | ? **eliminated** (delegate validation + trust md5, win #2) | this PR |

148

149 ### 4.5 The async copy-to-bucket durability model (spec ? needed *only* if a future

upload-onto-L4 path is ever adopted)

150

151 The recommended A? design ****has no async-copy step**** (the bucket copy is the ***upload***, produced before compute). This section specs the model the epic asked for, so that if the owner overrides \$8 and adopts upload-onto-L4, the durability semantics are pinned rather than improvised:

152

153 - ****When the mirror runs:**** kick a background ``storage.put(local ? gs:///uploads/?)`` ****immediately on upload-complete**** (not "during segmentation" ? starting it later only lengthens the exposure window; there's no compute reason to delay it). Run it concurrently with validation/detect.

154 - ****Failure/resume:**** retry with capped exponential backoff; the mirror is ****idempotent**** (same object key = the md5). On worker preemption before the mirror confirms, the run is ****lost**** ? the job must fail with a ***retryable*** status so the client can re-submit (the source is still on the client). ****Never**** report success until the mirror is ****confirmed**** (object exists + size/md5 match).

155 - ****Exposure window:**** upload-complete ? mirror-confirmed, the ****only**** durable copy is L4 local PD. Window length ? mirror wall (1.83 GiB intra-region ? seconds?tens of seconds ``[design]``). A preemption in-window = re-upload. ****This window is the entire reason A? is preferred**** ? A?'s window is ***zero***.

156 - ****Ack contract:**** the client keeps the source until the API returns ``mirror_confirmed:true``; only then may it release it. (A? needs no such contract ? bucket-first.)

157

158 **## 5. Phased migration ? every step a config flip with a bucket-mediated fallback**

159

160 Built on #521's ``resolve_phase_placement(cfg, phase)`` so ****placement is config, not code****. Each phase is independently reversible (``control_plane`` ? ``cloud_run_l4``) and falls back to in-process on a non-cloud box.

161

162 | Phase | Change | Mechanism | Fallback | Status |

163 |---|---|---|---|---|

164 | ****P0**** | ***Today*** ? detect on L4; validate+stitch on CP | ``compute_target=cloud_run`` | in-process | shipped ``[verified]`` |

165 | ****P1**** | ****Stitch ? L4**** | ``phase_placement.compile=cloud_run_l4`` (+ ``cloud_stitch_encoder``) | in-process stitch | ****#521 (open)**** |

166 | ****P2a**** | ****Drop the CP re-hash**** (trust submit-time md5) | ``deployment.trust_submit_video_id`` (default on) | re-hash on the drain | ****this PR (win #2)**** |

167 | ****P2b**** | ****Validation ? L4**** (CP stops running validators) | ``phase_placement.validation=cloud_run_l4`` (guards: requires cloud segment) | CP validates | ****#521 (open)**** |

168 | ****P2 net**** | ****CP never downloads the file**** ? the ~30 s gap dies; movement 3x?2x | P2a ? P2b together | either half alone degrades gracefully | needs #521 ***** win #2 merged |

169 | ****P3**** | Worker serializes its ****validation verdict**** back (per-validator results + #518 cache warm), so a worker-side reject renders the same SPA detail as a CP reject | new ``report.json``/marker field + CP ingest | worker reject = rc?0 + message (today) | ****follow-up**** (see \$6) |

170 | ****P4**** | ***If and only if*** sustained load crosses the \$4.2 breakeven | persistent worker (VM/Service) | scale-to-zero Job | ****owner-gated, new ADR**** |

171

172 ****The "collapse onto the L4" the epic asks for = P1 + P2, both config flips on #521's knob + this PR's win #2.**** No rearchitecture, no new infra, fully reversible.

173

174 **## 6. Honest limits ? what this does NOT do / does NOT prove**

175

176 - ****A green suite does not prove the cloud path**** ? the tests are fully-mocked/offline (a fake ``CloudRunJobClient``). P2's "CP never downloads" and the idle-GPU/round-trip numbers are validated ****in the cost model + code paths****, not on live infra. A live CUJ on rev ? ``00020`` (owner-side, D17 budget) is required to ***confirm*** the gap is gone and the movement is 2x.

177 - ****Network-egress "free intra-region" is ``[researched]``, not live-verified**** ? general web egress is blocked here, so same-region GCS?Cloud Run being un-billed-as-internet-egress rests on documented GCP behavior, not a fetched invoice. If it were ***not*** free, A? is ***still*** cheapest (it moves the fewest bytes); the conclusion is robust to this uncertainty.

178 - ****Win #2 changes nothing when no trusted id is present**** ? local/appliance uploads and non-GCS direct-URI submits (``submit_time_video_id`` ? ``None``) still re-hash. The win applies to

the GCS cloud-serving path (the CUJ case).

179 - ****P3 (worker validation-verdict serialization) is a real gap #521 leaves open****
`[verified]`: today a worker-side validation reject surfaces as ``rc?0`` + a message, but the per-validator ``ValidationResult`` list (``result['results']`` the SPA renders) and the #518 ``validation_cache`` warm are ****not**** reconstructed on the CP. Until P3, moving validation to the L4 (P2b) trades richer verdict UX + cache-warming for the CPU offload. Called out so P2b isn't flipped on blind.

180 - ****SQLite is the residual CP scaling limit**** ? the thin router is stateless ***except*** the SQLite DB; true horizontal scale-out needs Postgres (out of scope; noted).

181 - ****The #513 asymmetry reverses under P2b**** `[verified]`: worker-side validators run at model-default thresholds unless the CP forwards ``validation.*`` as ``--set``. #521's validation-delegation path must forward those (or re-inherit the divergence that caused the #513 incident). Flagged for the #521 review, not owned here.

182

183 **## 7. Deliverables & progress tracker ? source of truth**

184

185 Legend: ? Todo · ? In progress · ? Done · ? Gated. One small PR per row, branched from ``origin/master``, no stacking.

186

ID	Deliverable	Depends on	Gated?	Status	PR
P0	**This design doc** (resolves topology A/B/C; recommends A?; specs durability + phased migration)	?	owner-review	?	(this PR) #524
P1	**Win #2** ? drop the CP re-hash; trust submit-time md5; <code>`deployment.trust_submit_video_id`</code> (default on), reversible	?	No	?	(this PR) #524
P2	**Win #1** ? validation-on-L4 (<code>`phase_placement.validation`</code>) #521	No	?	?	?
delivered by #521 ? not duplicated here #521					
P3	**CP-download-avoidance measured live** ? confirm the ~30 s gap is gone + movement is 2x on rev ? <code>`00020`</code> #521 + P1 merged	?	owner-side (live spend, D17)	?	?
P4	**Worker validation-verdict serialization** (\$6 gap) so P2b keeps rich SPA verdict + #518 cache #521	No	?	follow-up	?
P5	**Persistent-GPU re-evaluation** only if load crosses \$4.2 breakeven volume ? new ADR	?	?	?	?

195

196 **## 8. Open questions ? owner / external**

197

198 ****Blocking (owner sign-off):****

199 1. ****Do you accept rejecting "upload directly onto the L4" (L5)?**** The epic proposed it; this doc recommends against it with the \$4.2/\$4.3 idle-GPU-vs-free-LAN-pull numbers. A? delivers the same "collapse onto the L4" outcome, cheaper and more durable, via config. ***If you still want upload-onto-L4*** (e.g. for a future no-bucket edge deployment), \$4.5 specs its durability model ? but it needs a Service-with-GPU or VM (a D16(a)/D7 reversal ? new ADR).

200 2. ****Flip P2b (validation?L4) before or after P3 (verdict serialization)?**** P2b alone offloads CPU but degrades the reject-UX + skips #518 cache-warm (\$6). Recommend: land P3 first, then P2b.

201

202 ****Nice-to-know:****

203 3. Is pilot volume expected to cross the \$4.2 breakeven within the planning horizon? (Decides whether P5 is ever live.)

204

205 **## 9. Definition of done**

206

- [x] Design doc resolves the topology question (A/B/C scored) with a clear recommendation (A?) + quantified cost/latency/reliability. `[this PR]`
- [x] Minimal control-plane surface defined (\$4.4). `[this PR]`
- [x] Async copy-to-bucket durability model specified with failure/resume + exposure window (\$4.5). `[this PR]`
- [x] Phased migration laid out on #521's ``phase_placement``, each step a config flip with a bucket-mediated fallback (\$5). `[this PR]`
- [x] Win #2 landed (config-guarded, reversible, tests green). `[this PR]`
- [] Win #1 confirmed delivered by #521 (not duplicated). `[cross-PR]`
- [] Live CUJ confirms the gap is gone + movement is 2x (P3, owner-side). `[gated]`
- [] Owner signs off on L5 (reject upload-onto-L4) ? flip Status ? IN PROGRESS/DONE.

215

216 **## 10. References**

```

217
218     **Internal code anchors `[verified]`:** `backend/api/uploads.py` (streamed md5),
`backend/orchestration.py:220-239` (`submit_time_video_id`, #484) / `:741-763` (re-hash) /
`_maybe_dispatch_remote`, `backend/api/jobs.py:80-153` (submit-time cache already trusts the
md5), `backend/api/job_worker.py:122-193` (`_job_to_request`/`_run_claimed_job`),
`backend/compute/cloud_run.py` (dispatch + bucket-poll), `backend/config/placement.py` (#521
`resolve_phase_placement`), `backend/worker/__main__.py` (worker validation + `--skip-
validation`).
219     **Internal docs:** [`PHASE_PLACEMENT.md` (#521)](https://github.com/Khelsutra/badminton-highlight-indexer/blob/feat/521-compile-on-l4/docs/COMPUTE\_DECOUPLED\_SERVING/PHASE\_PLACEMENT.md), [README.md](./README.md) (ADR-014,
D4/D7/D16), `runlogs/DEPLOY_REPORT_cloud-serving_validation-latency_2026-07-07.md` §9
(evidence).
220     **External `[verified]` rates:** GCP Cloud Billing Catalog API
(`cloudbilling.googleapis.com`, asia-southeast1, USD, 2026-07-08). **`[researched]`:** same-
region GCS?Cloud Run egress treatment (not live-fetchable here).
221
222     ---
223
224     ### Changelog
225     - `2026-07-08` ? initial draft: topology A/B/C resolved (recommend A? = co-located Jobs
+ bucket upload; reject persistent-GPU + upload-onto-L4 with a cost model); durability + phased
migration on #521's `phase_placement`; win #2 landed alongside.
226

```

5. user (2026-07-08T06:36:35.963Z)

```

diff --git a/backend/orchestration.py b/backend/orchestration.py
index 8b7851e..7769e4e 100644
--- a/backend/orchestration.py
+++ b/backend/orchestration.py
@@ -239,6 +239,20 @@ def submit_time_video_id(input_ref, cfg) -> Optional[str]:
     return None

+def _looks_like_video_id(value: Optional[str]) -> bool:
+    """True iff `value` is a well-formed `video_id` ? the 32-char lowercase hex MD5 that
+    `compute_video_id` produces (`backend/utils/hashing.py`). Guards the #524 win-#2 trust
+    path:
+    + a trusted id is only reused in place of a re-hash when it has the EXACT shape the drain
+    would
+    + have computed, so a malformed / truncated / wrong-encoding id can never become the DB key ?
+    + it degrades to a real hash instead. `submit_time_video_id` already normalizes the gs://
+    + base64 md5 to this hex form (#484, `backend/storage/gcs.py::_md5_hex`)."""
+    return (
+        isinstance(value, str)
+        and len(value) == 32
+        and all(c in "0123456789abcdef" for c in value)
+    )
+
def _cached_process_payload(
    video_id: str, validation_results, segments, requested_compute
) -> Dict[str, Any]:
@@ -706,7 +720,7 @@ def _maybe_dispatch_remote(

def _execute_processing(
-    config=None, db=None, req=None, progress=None
+    config=None, db=None, req=None, progress=None, trusted_video_id=None
) -> Dict[str, Any]:
    """The full hash ? validate ? segment ? report pipeline for one video.

@@ -716,6 +730,15 @@ def _execute_processing(
    test code that calls `_execute_processing(req)` still works.

```

```

    ``progress`` is an optional callable(stage: str) used to surface coarse job progress.
+
+   ``trusted_video_id`` is the SERVER-DERIVED submit-time md5 the drain already knows for this
+   job (``jobs.video_id`` ? ``submit_time_video_id``, #484) ? byte-identical to
+   ``compute_video_id`` and already trusted for the submit-time cache probe. When present (and
+   ``deployment.trust_submit_video_id`` is on), it is reused so the drain SKIPS re-hashing the
+   multi-GB clip it just fetched (#524 win #2); absent / malformed ? the drain hashes as
before.
+   It is a keyword-only, internal channel ? NOT a client-supplied request field (a caller's
+   ``/api/process`` body cannot inject it; the submit hook recomputes the authoritative id).
+
    Raises on unexpected internal errors ? the caller records those as a job failure
    """
    import backend.main as _main # call-time seam: main's globals/model stay patchable
@@ -748,7 +771,31 @@ def _execute_processing(
        local_video = storage.acquire_local_input(
            req.video_path, getattr(config, "storage", None)
        )
-       video_id = _main.compute_video_id(local_video)
+       # #524 win #2: trust the SERVER-DERIVED submit-time md5 instead of re-hashing the whole
+       # (often multi-GB) clip we just fetched. The trusted id is byte-identical to
+       # compute_video_id (same bytes, same MD5 ? #484) and is ALREADY the value the submit-
time
+       # cache probe trusts, so reusing it here is consistent, not a new trust assumption.
Gated by
+       # deployment.trust_submit_video_id (default on) and only honoured when the id is well-
formed
+       # (_looks_like_video_id) ? any absence/malformation degrades to a real hash. The fetch
above
+       # is unchanged; only the redundant hash is skipped.
+       trust_submitted = bool(
+           getattr(
+               getattr(config, "deployment", None), "trust_submit_video_id", True
+           )
+       )
+       # STRICT: trust only an id in the EXACT form the drain would have produced (32-char
+       # lowercase hex, per compute_video_id / submit_time_video_id). No normalization ? a
+       # differently-cased or malformed id is a signal something is off, so re-hash rather
than
+       # silently coerce it into the DB key.
+       candidate = trusted_video_id or ""
+       if trust_submitted and _looks_like_video_id(candidate):
+           video_id = candidate
+           logger.info(
+               "[API] trusting submit-time video_id=%s ? skipped the drain re-hash (#524).",
+               video_id,
+           )
+       else:
+           video_id = _main.compute_video_id(local_video)
    except Exception:
        storage.cleanup_acquired_local_input(
            req.video_path, local_video, getattr(config, "storage", None)

```

6. user (2026-07-08T06:36:39.597Z)

```

738         multi-GB clip it just fetched (#524 win #2); absent / malformed ? the drain hashes
as before.
739         It is a keyword-only, internal channel ? NOT a client-supplied request field (a
caller's
740         ``/api/process`` body cannot inject it; the submit hook recomputes the authoritative
id).
741
742         Raises on unexpected internal errors ? the caller records those as a job failure
743         """
744         import backend.main as _main # call-time seam: main's globals/model stay patchable

```

```

745
746     # Backward-compatible shim: if called as _execute_processing(req) (old pattern),
747     # shift positional args ? req arrives in position 0 (config), progress in position 1
748     (db).
749     if config is not None and not isinstance(config, AppConfig):
750         # Old signature: _execute_processing(req, progress=None)
751         if req is None and db is not None and isinstance(db, _main.ProcessRequest):
752             req = db
753             db = None
754         elif req is None and isinstance(config, _main.ProcessRequest):
755             req = config
756             config = None
757     # Resolve config/db from module globals when not passed explicitly
758     if config is None:
759         config = _main.global_config
760     if db is None:
761         db = _main.db_instance
762
763     notify = progress or (lambda stage: None)
764
765     notify("hashing")
766     # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
767     (identity ?
768     # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
769     original
770     # req.video_path (the source URI) is kept for the DB record + report; only the file-
771     reading
772     # consumers (hash / validators / segmenter) use the resolved local path.
773     local_video = ""
774     try:
775         local_video = storage.acquire_local_input(
776             req.video_path, getattr(config, "storage", None)
777         )
778         # #524 win #2: trust the SERVER-DERIVED submit-time md5 instead of re-hashing
779         the whole
780         # (often multi-GB) clip we just fetched. The trusted id is byte-identical to
781         # compute_video_id (same bytes, same MD5 ? #484) and is ALREADY the value the
782         submit-time
783         # cache probe trusts, so reusing it here is consistent, not a new trust
784         assumption. Gated by
785         # deployment.trust_submit_video_id (default on) and only honoured when the id is
786         well-formed
787         # (_looks_like_video_id) ? any absence/malformation degrades to a real hash. The
788         fetch above
789         # is unchanged; only the redundant hash is skipped.
790         trust_submitted = bool(
791             getattr(
792                 getattr(config, "deployment", None), "trust_submit_video_id", True
793             )
794         )
795         # STRICT: trust only an id in the EXACT form the drain would have produced
796         (32-char
797         # lowercase hex, per compute_video_id / submit_time_video_id). No normalization
798         ? a
799         # differently-cased or malformed id is a signal something is off, so re-hash
800         rather than
801         # silently coerce it into the DB key.
802         candidate = trusted_video_id or ""
803         if trust_submitted and _looks_like_video_id(candidate):
804             video_id = candidate
805             logger.info(
806                 "[API] trusting submit-time video_id=%s ? skipped the drain re-hash
807                 (#524).",
808                 video_id,
809             )

```

```

797         else:
798             video_id = _main.compute_video_id(local_video)
799     except Exception:
800         storage.cleanup_acquired_local_input(
801             req.video_path, local_video, getattr(config, "storage", None)
802         )
803         raise
804     try:
805         existing_video = db.get_video(video_id)
806     except Exception:
807         storage.cleanup_acquired_local_input(

```

7. assistant (2026-07-08T06:37:32.406Z)

I've verified all the specific claims against the actual committed code and doc. Let me confirm the pre-insertion line numbers to establish the root cause.

8. user (2026-07-08T06:37:33.608Z)

```

=== OLD (pre-PR) orchestration.py around the re-hash ===
745     # consumers (hash / validators / segmenter) use the resolved local path.
746     local_video = ""
747     try:
748         local_video = storage.acquire_local_input(
749             req.video_path, getattr(config, "storage", None)
750         )
751         video_id = _main.compute_video_id(local_video)
752     except Exception:

```

9. assistant (2026-07-08T06:37:50.980Z)

Verified. The root cause is now definitive.

****Pre-PR (HEAD) `backend/orchestration.py`:**** line 748 = `local\_video = storage.acquire\_local\_input(`, line 751 = `video\_id = \_main.compute\_video\_id(local\_video)`. These are exactly the anchors the doc cites in §0.

****Post-PR (committed working tree):**** this PR inserts ~27 lines of win-#2 trust logic (lines 774-796) before the re-hash. That shifts:

- `acquire\_local\_input` ? line 771 (verified: line 771 = `local\_video = storage.acquire\_local\_input(`)
- `compute\_video\_id` re-hash ? line 798 (verified: line 798 = `video\_id = \_main.compute\_video\_id(local\_video)`)

Meanwhile the committed lines the doc points at are unrelated code:

- Line 748 = `if config is not None and not isinstance(config, AppConfig):` (the compat shim)
- Line 751 = `req = db`
- Lines 741-763 (§10's re-hash range) = docstring tail + compat shim + config/db resolution ? the re-hash is nowhere in that range; it is at 798.

The doc (`L4\_PRIMARY\_TOPOLOGY.md`) is a NEW file added in this same PR, and its anchors were written against the pre-insertion line numbers. Since the doc explicitly tags these anchors `[verified]` (line 7: "checked against code/runlog") and stakes its credibility on that tag, shipping anchors that resolve to unrelated code in the same commit is a genuine same-commit accuracy defect, not a style nit. §0 (line 23) and §10 (line 218) both point a reader following a `[verified]` anchor at the wrong code. The `:220-239` anchor for `submit\_time\_video\_id` is unaffected (the insertion is after it) and remains correct.

10. user (2026-07-08T06:38:00.615Z)

Structured output provided successfully